

AI 辅助编程入门

工具 · 能力 · 提示词工程

一个老牌技术社区的衰落，揭示了开发者世界正在发生的范式转变.....

南京大学 · 《软件工程与计算 II》 · 2026 春季学期

模块	关键问题
AI4Coding 是什么	工具能做什么，不能做什么
四大主流工具	GitHub Copilot · Cursor · 通义灵码 · Claude Code
提示词工程	如何让 AI 输出高质量代码

开场案例：Stack Overflow 的危机（2022—2023）

"我们所有人都是在 Stack Overflow 上长大的。" ——无数程序员的共同记忆

Stack Overflow：2008 年创立，全球最大的编程问答社区

时间线	事件
2022 年	月活跃页面浏览量 140 亿次 ，鼎盛时期
2022-11	ChatGPT 发布，开发者开始转向 AI 问答
2023-03	GitHub Copilot X 发布，代码补全进化为对话
2023 年	Stack Overflow 流量同比下滑 约 20%
2023-05	Stack Overflow 宣布裁员 28% （约 100 人）

根因分析：信息获取方式的范式转变

根本原因：开发者的信息获取方式，正在从 "搜索 + 社区问答" 切换为 "对话 + 即时生成"

旧模式："我遇到了一个 Python bug，去 Google 搜，找到 Stack Overflow 答案，复制粘贴"

新模式："我遇到了一个 Python bug，在 IDE 里直接问 AI，AI 帮我定位并修复"

数据印证 (Stack Overflow 2023 年度开发者调查，70,000+ 受访者)：

- **44%** 的开发者已在工作流中使用 AI 工具
- **62%** 的人认为 AI 工具提升了生产效率
- 最常用的 AI 工具：ChatGPT (83%)、GitHub Copilot (56%)

💡 这不是威胁，这是工具升级的机会——会使用 AI 的工程师，正在取代不会使用 AI 的工程师

本讲路线图

模块	核心问题	时长
PART 1	AI 辅助编程是什么？能做什么？	~15 min
PART 2	四大主流工具怎么上手？	~20 min
PART 3	如何写出有效的提示词？	~15 min

本讲结束时，你应该能回答："AI 工具在软件开发中的边界在哪里？"

PART 1

| AI4Coding 是什么?

从代码补全到对话驱动开发

AI 辅助编程的演进历程

阶段	代表工具	核心交互方式	能力边界
2021	Tab 补全 GitHub Copilot	按 Tab 接受建议	单函数级
2022	对话生成 ChatGPT	聊天输入自然语言	文件级
2023	项目理解 Cursor / Lingma	扫描整个项目上下文	项目级
2024	自主 Agent Devin / Claude	自主执行多步任务	任务级

我们当前处于 "AI 辅助人"→"人辅助 AI" 的过渡阶段——
工具已经从"按 Tab 补全"进化为"对话驱动整个项目"

主流工具生态对比

工具	定位	集成方式	核心优势
GitHub Copilot	微软 / OpenAI 官方	VS Code / IDEA 插件	代码补全、Inline Chat
Cursor	AI-First IDE	独立 IDE（基于 VS Code）	对话驱动、多文件理解
通义灵码	阿里巴巴	VS Code / IDEA 插件	国内网络友好、企业级场景
ChatGPT / Claude	通用 AI	Web / API	架构设计、解释、代码审查

本课程主要使用 **Cursor** 和 **通义灵码**——环境友好，能力全面

AI 编程的六大核心能力

能力	描述	示例提示词
代码生成	从自然语言描述生成代码	"实现一个二分查找函数"
代码解释	理解并解释代码逻辑	"解释这段代码做了什么"
Bug 修复	定位并修复错误	"找出并修复这段代码的 bug"
代码注释	自动生成文档注释	"为这个函数生成 JavaDoc"
测试生成	自动生成单元测试	"为这个方法生成测试用例"
代码重构	优化代码结构与质量	"重构这段代码，提高可读性"

💡 本课程 07 讲（Java Web 开发）的 Spring Boot / MyBatis 代码，以上 6 种能力都可以用 AI 辅助完成

PART 2

四大工具上手

GitHub Copilot · 通义灵码 · Cursor · Claude Code

GitHub Copilot: 核心功能详解

功能一：内联代码补全

- 输入函数注释或签名，按 `Tab` 接受 AI 建议
- 按 `Esc` 拒绝，`Alt+]` 查看下一个建议

功能二：Inline Chat (`Ctrl+I`)

- 选中代码 → `Ctrl+I` → 直接用自然语言修改

功能三：Copilot Chat 侧边栏

- 解释代码： `/explain`
- 修复 Bug： `/fix`
- 生成测试： `/tests`

```
# 实现一个函数，对字符串列表按长度排序，相同长度的按字母顺序排列
def sort_strings(strings: list[str]) -> list[str]:
    # ↓ Copilot 在此处自动补全 ↓
```

通义灵码（Lingma）：安装与使用

安装：VS Code 扩展市场搜索 **Lingma**（Alibaba Cloud）或 IDEA 插件市场

核心特色：

功能	说明
智能续写	输入代码时实时弹出补全建议
自然语言生成类	输入注释描述，自动生成完整 Java 类
跨文件感知	理解整个项目的上下文（如 Mapper.xml 与接口对应）
中文提示词	支持纯中文描述，对国内框架（Spring Boot 等）理解更好

✅ 推荐场景：Java 后端开发（Spring Boot + MyBatis）使用通义灵码，前端开发使用 Cursor

Cursor：对话驱动开发

安装：从 cursor.sh 下载，基于 VS Code，快捷键完全兼容

快捷键	功能	典型用法
Tab	接受 AI 代码建议	自动补全当前行
Ctrl+K	内联编辑	选中代码 → AI 直接修改
Ctrl+L	打开 AI 对话面板	对话式问答
Ctrl+Shift+L	添加代码到对话上下文	选中后拖入对话窗口
Ctrl+Enter	扫描整个项目	跨文件理解，回答架构问题

核心优势： Ctrl+Enter 让 AI 理解整个项目，而不只是当前文件——这是 Cursor 超越插件类工具的关键

Claude Code：终端里的自主编程 Agent

安装： `npm install -g @anthropic-ai/claude-code`，在项目目录下执行 `claude`

与 Copilot / Cursor 的本质区别：

	Copilot / Cursor	Claude Code
工作方式	编辑器内嵌，逐行补全	终端 Agent，自主完成多步任务
操作粒度	函数 / 文件级	整个项目级（读写文件、执行命令、管理 Git）
交互模式	你写、AI 补	委托任务，AI 规划并执行全流程
最强场景	边写边补全	重构、添加功能、Debug、生成测试、代码审查

典型用法：

```
# 进入项目，直接用自然语言描述任务
$ claude
> 把所有 API 接口加上统一的错误处理，遵循现有的代码风格
> 找出项目中所有潜在的 SQL 注入风险并修复
> 为 UserService 生成完整的单元测试，覆盖边界条件
```

💡 **核心优势：** Claude Code 能读懂整个代码库的上下文，像一个懂你项目的高级工程师——不只是补全代码，而是真正理解架构后再动手

动手实践（5 分钟）

任务：体验 AI 生成代码的完整流程

1. 打开 Cursor 或安装通义灵码，新建 `fibonacci.py`
2. 输入注释： `# 实现斐波那契数列，支持递归和迭代两种方式`
3. 让 AI 生成代码，运行验证输出结果
4. **追问**：让 AI 解释两种实现方式的时间复杂度差异
5. **挑战**：让 AI 自动生成 3 个测试用例，包含边界值（ $n=0$, $n=1$, $n=10$ ）

✅ 成功标志：AI 生成的代码能正确运行，且你能看懂每一行在做什么

⚠️ 警告：如果 AI 生成的代码你完全看不懂，不要直接使用——这是技术债的开端

PART 3

| 提示词工程

让 AI 输出你真正想要的代码

好提示词的四要素

公式： [角色] + [任务] + [约束] + [格式]

✗ 差的提示词：
"帮我修 bug"

✓ 好的提示词：
"作为一名熟悉 Java Spring Boot 的后端工程师， [角色]
分析以下代码中导致空指针异常的根本原因， [任务]
仅修改出错的方法，不要改动其他代码， [约束]
给出修复后的完整方法代码，并用一句话解释原因。" [格式]

核心原则：约束越精确，AI 的输出越可控

AI 不会自动猜测你的意图——它会忠实地执行你给的指令，哪怕指令有歧义

10 个实用提示词：精准控制类

#	提示词模板	适用场景
1	"仅修复此问题，不要修改其他代码"	防止 AI 改动范围过大
2	"请使用最小修改来解决此问题"	保持现有代码风格一致
3	"换个思路来解决此问题"	当前方案性能或逻辑不满意
4	"仔细阅读全文，检查是否还有其他同类问题"	让 AI 做全局自我审查
5	"请为我梳理当前代码的整体逻辑"	理解他人代码或遗留代码

最常用：No.1 和 No.2 能避免 AI 生成"牵一发动全身"的修改

10 个实用提示词：代码质量类

#	提示词模板	适用场景
6	"请提供详细的解释和行内代码注释"	生成可维护的文档
7	"生成测试用例来验证此更改，包含正常路径和异常路径"	自动化测试覆盖
8	"增加详细的日志输出，方便追溯错误"	调试辅助
9	"请简化代码逻辑，去除冗余部分"	代码瘦身
10	"优化代码结构，使其更加模块化，方便单元测试"	重构升级

💡 No.7 组合 No.8 特别有效：让 AI 生成带日志的测试用例，调试时事半功倍

🗨️ 停下来想一想

"AI 工具让编程变容易了——那我们还需要扎实地学编程基础吗？"

正方观点：AI 能生成代码，只需会描述需求就够了

反方观点（推荐思考方向）：

- AI 生成的代码需要**人来审查**——没有基础，看不懂，也发现不了错误
- 你的提示词质量 $\propto$ 你对问题的理解深度 (**Garbage in, garbage out**)
- 在你不理解的代码上继续让 AI 叠加功能，技术债会**指数级增长**
- AI 在边界条件、并发安全、性能优化等方面经常犯错——需要你来把关

结论：基础越扎实 $\rightarrow$ 提示词越精准 $\rightarrow$ AI 输出越高质量 $\rightarrow$ 你的生产效率越高

课堂总结：人与 AI 的正确分工

你的职责

- 理解业务需求
- 设计系统架构
- 审查 AI 的输出
- 评估代码质量与安全性
- 处理边界条件与异常场景
- 做最终的技术决策

AI 的职责

- 快速生成代码草稿
- 自动补全重复模式代码
- 解释不熟悉的代码逻辑
- 生成测试用例初稿
- 提供多种实现思路供选择
- 执行重构和格式化任务

AI 是你的 Pair Programmer，不是你的替代品

那些最高效地使用 AI 工具的工程师，往往是技术基础最扎实的人

作业（二选一）

选题 A：工具探索报告

1. 安装并深度体验 Cursor 或通义灵码中的任意一个
2. 用该工具完成一个自选小任务（排序算法 / 链表操作 / 简单 REST API)
3. 记录至少 **3 轮**提示词迭代过程，总结工具的优缺点
4. 提交：迭代记录截图 + 300 字总结

选题 B：提示词对比实验

1. 选取一道 LeetCode 简单题（或自选算法题）
2. 分别用**差的提示词**和**好的提示词**各问一次 AI
3. 对比输出质量：代码正确性、注释完整度、边界处理、代码风格
4. 提交：两次对话截图 + 分析报告（描述差异的根本原因，300 字）

三分钟回顾

问题	答案
AI 编程工具的六大核心能力?	生成 · 解释 · Debug · 注释 · 测试 · 重构
Cursor 扫描整个项目的快捷键?	<code>Ctrl+Enter</code>
好提示词的四要素?	角色 + 任务 + 约束 + 格式
Stack Overflow 流量为何下滑?	AI 改变了开发者信息获取方式
有了 AI 为什么还要学基础?	基础决定提示词质量; AI 输出需人工审查
最能防止 AI 改动范围失控的提示词?	"仅修复此问题, 不要修改其他代码"

下节预告

第 8-02 讲：开发工具链与 Git 工作流

2012 年，一家公司因为**没有版本控制**，在 45 分钟内损失了 **4.4 亿美元**.....

南京大学 · 《软件工程与计算 II》 · 2026 春季学期

第 8-01 讲 AI 辅助编程入门

本讲核心收获

- AI 编程工具生态: Copilot · Cursor · 通义灵码的定位与差异
- 六大核心能力: 生成/解释/Debug/注释/测试/重构
- 提示词工程: 四要素公式 + 10 个实用模板

工具是手段，思维是根本